VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

ADITYA A(1BM24CS017)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by ADITYA A(1BM24CS017), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026
. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.

C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1a

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & non-preemptive)

```
#include <stdio.h>
typedef struct
{
   int at, bt, ct, tat, wt;
   int done, remaining_bt, priority;
} Process;
void fcfs(Process p[], int n)
{
   int cp = 0, curr_time = 0
   for (int i = 0; i < n; i++)
      p[i].done = 0;
   while (cp != n)
   {
      int smallest_at = 1e9, idx = -1;
      for (int i = 0; i < n; i++)
      {
         if (p[i].at < smallest_at && !p[i].done && p[i].at <= curr_time)
         {
            smallest_at = p[i].at;
            idx = i;
         }
      }
      if (idx == -1)
```

```
        {
            curr_time++;
            continue;
        }
        curr_time += p[idx].bt;
        p[idx].ct = curr_time;
        p[idx].done = 1;
        cp++;
    }
}
void sjf(Process p[], int n)
{
    int cp = 0, curr_time = 0;
    for (int i = 0; i < n; i++)
    {
        p[i].done = 0;
    }
    while (cp != n)
    {
        int smallest_bt = 1e9, idx = -1;
        for (int i = 0; i < n; i++)
        {
            if (!p[i].done && p[i].at <= curr_time && p[i].bt <= smallest_bt)
            {
                if (p[i].bt == smallest_bt && p[i].at > p[idx].at)
                {
                    continue;
                }
```

```
            smallest_bt = p[i].bt;
            idx = i;
        }
    }
    if (idx == -1)
    {
        curr_time++;
        continue;

    curr_time += p[idx].bt;
    p[idx].ct = curr_time;
    p[idx].done = 1;
    cp++;
  }
}
void srtf(Process p[], int n)
{
  int cp = 0, curr_time = 0;
  for (int i = 0; i < n; i++)
  {
    p[i].remaining_bt = p[i].bt;
  }
  while (cp != n)
  {
    int smallest_rt = 1e9, idx = -1;
    for (int i = 0; i < n; i++)
    {
      if (p[i].at <= curr_time && p[i].remaining_bt > 0 && p[i].remaining_bt <= smallest_rt)
      {
```

```
            if (p[i].remaining_bt == smallest_rt && p[i].at > p[idx].at)
            {
                continue;
            }
            smallest_rt = p[i].remaining_bt;
            idx = i;
        }
    }
    if (idx == -1)
    {
        curr_time++;
        continue;
    }
    p[idx].remaining_bt--;
    curr_time++;
    if (p[idx].remaining_bt == 0)
    {
        p[idx].ct = curr_time;
        cp++;
    }
  }
}
void calc_avg(Process p[], int n, float *avg_tat, float *avg_wt)
{
  int tot_tat = 0, tot_wt = 0;
  printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
  for (int i = 0; i < n; i++)
  {
    p[i].tat = p[i].ct - p[i].at;
```

```
        p[i].wt = p[i].tat - p[i].bt;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
        tot_tat += p[i].tat;
        tot_wt += p[i].wt;
    }
    *avg_tat = (float)tot_tat / n;
    *avg_wt = (float)tot_wt / n;
}
int main()
{
    float avg_tat, avg_wt;
    Process p[5];
    p[0].at = 0;
    p[0].bt = 3;
    p[0].priority = 5;
    p[1].at = 2;
    p[1].bt = 2;
    p[1].priority = 3;
    p[2].at = 3;
    p[2].bt = 5;
    p[2].priority = 2;
    p[3].at = 4;
    p[3].bt = 4;
    p[3].priority = 4;
    p[4].at = 6;
    p[4].bt = 1;
    p[4].priority = 1;
    fcfs(p, 5);
    sjf(p, 5);
```

```
    srtf(p, 5);
    calc_avg(p, 5, &avg_tat, &avg_wt);
    printf("Avg TAT: %.2f\n", avg_tat);
    printf("Avg WT: %.2f\n", avg_wt);
}
```

Result:

```
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017> cd "c:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017\" ; if ($?
) { gcc program1.c -o program1 } ; if ($?) { .\program1 }
FCFS
Process AT      BT      CT      TAT     WT
1       2       1       12      10      9
2       1       5       11      10      5
3       4       1       16      12      11
4       0       6       6       6       0
5       2       3       15      13      10
Avg TAT: 10.20
Avg WT: 7.00

SJF
Process AT      BT      CT      TAT     WT
1       2       1       8       6       5
2       1       5       16      15      10
3       4       1       7       3       2
4       0       6       6       6       0
5       2       3       11      9       6
Avg TAT: 7.80
Avg WT: 4.60

SRTF
Process AT      BT      CT      TAT     WT
1       2       1       3       1       0
2       1       5       11      10      5
3       4       1       5       1       0
4       0       6       16      16      10
5       2       3       7       5       2
Avg TAT: 6.60
Avg WT: 3.40
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017>
```

Program 1b:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→ Priority (pre-emptive & Non-pre-emptive)
→Round Robin

```
#include<stdio.h>
typedef struct
{
   int at, bt, ct, tat, wt;
   int done, remaining_bt, priority;
} Process;
void priority_np(Process p[], int n)
{
   int cp = 0, curr_time = 0;
   for (int i = 0; i < n; i++)
   {
      p[i].done = 0;
   }
   while (cp != n)
   {
      int highest_pr = 1e9, idx = -1;
      for (int i = 0; i < n; i++)
      {
         if (!p[i].done && p[i].at <= curr_time && p[i].priority <= highest_pr)
         {
            if (p[i].priority == highest_pr && p[i].at > p[idx].priority)
            {
               continue;
            }
            highest_pr = p[i].priority;
            idx = i;
         }
```

```
        }
        if (idx == -1)
        {
            curr_time++;
            continue;
        }
        curr_time += p[idx].bt;
        p[idx].ct = curr_time;
        p[idx].done = 1;
        cp++;
    }
}
void priority_p(Process p[], int n)
{
    int cp = 0, curr_time = 0
    for (int i = 0; i < n; i++)
    {
        p[i].remaining_bt = p[i].bt;
    }
    while (cp != n)
    {
        int highest_pr = 1e9, idx = -1;
        for (int i = 0; i < n; i++)
        {
            if (p[i].at <= curr_time && p[i].remaining_bt > 0 && p[i].priority <= highest_pr)
            {
                if (p[i].priority == highest_pr && p[i].at > p[idx].at)
                {
                    continue;
                }
                highest_pr = p[i].priority;
                idx = i;
```

```
            }
        }
        if (idx == -1)
        {
            curr_time++;
            continue;
        }
        p[idx].remaining_bt--;
        curr_time++;
        if (p[idx].remaining_bt == 0)
        {
            p[idx].ct = curr_time;
            cp++;
        }
    }
}
void round_robin(Process p[], int n, int tq)
{
    int cp = 0, curr_time = 0;
    for (int i = 0; i < n; i++)
    {
        p[i].remaining_bt = p[i].bt;
    }
    while (cp != n)
    {
        int found = 0;
        for (int i = 0; i < n; i++)
        {
            if (p[i].at <= curr_time && p[i].remaining_bt > 0)
            {
                found = 1;
                if (p[i].remaining_bt >= tq)
```

```
            {
                curr_time += tq;
                p[i].remaining_bt -= tq;
            }
            else
            {
                curr_time += p[i].remaining_bt;
                p[i].remaining_bt = 0;
                p[i].ct = curr_time;
                cp++;
            }
        }
    }
    if (!found)
    {
        curr_time++;
    }
  }
}
void calc_avg(Process p[], int n, float *avg_tat, float *avg_wt)
{
  int tot_tat = 0, tot_wt = 0;
  printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
  for (int i = 0; i < n; i++)
  {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    printf("%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    tot_tat += p[i].tat;
    tot_wt += p[i].wt;
  *avg_tat = (float)tot_tat / n;
  *avg_wt = (float)tot_wt / n;
```

```
}
int main()
{
    float avg_tat, avg_wt;
    Process p[5];
    p[0].at = 0;
    p[0].bt = 3;
    p[0].priority = 5;
    p[1].at = 2;
    p[1].bt = 2;
    p[1].priority = 3;
    p[2].at = 3;
    p[2].bt = 5;
    p[2].priority = 2;
    p[3].at = 4;
    p[3].bt = 4;
    p[3].priority = 4;
    p[4].at = 6;
    p[4].bt = 1;
    p[4].priority = 1;
    priority_np(p, 5);
    priority_p(p, 5);
    round_robin(p, r, 2);
    calc_avg(p, 5, &avg_tat, &avg_wt);
    printf("Avg TAT: %.2f\n", avg_tat);
    printf("Avg WT: %.2f\n", avg_wt);
}
```

Result:

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Process AT      BT      CT      TAT     WT
1       0       3       3       3       0
2       2       2       11      9       7
3       3       5       8       5       0
4       4       4       15      11      7
5       6       1       9       3       2
Avg TAT: 6.20
Avg WT: 3.20
```

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Process AT      BT      CT      TAT     WT
1       0       3       7       7       4
2       1       4       5       4       0
3       2       6       13      11      5
4       3       4       17      14      10
5       5       2       19      14      12
Avg TAT: 10.00
Avg WT: 6.20
```

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Process AT      BT      CT      TAT     WT
1       0       5       13      13      8
2       1       3       12      11      8
3       2       1       5       3       2
4       3       2       9       6       4
5       4       3       14      10      7
Avg TAT: 8.60
Avg WT: 5.80
```

Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#define MAX 10
typedef struct {
int pid, at, bt, rt, ct, tat, wt, queue;
} Process;
int main() {
Process p[MAX];
int n, i, time = 0, completed = 0;
int tq = 2;
printf("Enter number of processes: ");
scanf("%d", &n);
for (i = 0; i < n; i++) {
printf("\nP%d\n", i + 1);
p[i].pid = i + 1;
printf("Arrival Time: ");
scanf("%d", &p[i].at);
printf("Burst Time: ");
scanf("%d", &p[i].bt);
printf("Queue (1-System, 2-User): ");
scanf("%d", &p[i].queue);
p[i].rt = p[i].bt;
}
printf("\nGantt Chart:\n");
while (completed < n) {
int executed = 0;
for (i = 0; i < n; i++) {
```

```
        if (p[i].queue == 1 && p[i].rt > 0 && p[i].at <= time) {
                executed = 1;
                if (p[i].rt > tq) {
                        printf("P%d ", p[i].pid);
                        time += tq;
                        p[i].rt -= tq;
                } else {
                        printf("P%d ", p[i].pid);
                        time += p[i].rt;
                        p[i].rt = 0;
                        p[i].ct = time;
                        completed++;
                }
        }
}
if (!executed) {
        for (i = 0; i < n; i++) {
                if (p[i].queue == 2 && p[i].rt > 0 && p[i].at <= time) {
                        executed = 1;
                        printf("P%d ", p[i].pid);
                        p[i].rt--;
                        time++;
                        if (p[i].rt == 0) {
                                p[i].ct = time;
                                completed++;

                        break;
                }
        }
}
if (!executed) {
        time++;
```

```
}
}
float total_tat = 0, total_wt = 0;
printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt
total_tat += p[i].tat;
total_wt += p[i].wt;
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
      p[i].pid, p[i].at, p[i].bt,
      p[i].ct, p[i].tat, p[i].wt);
}
printf("\nAverage TAT = %.2f", total_tat / n);
printf("\nAverage WT = %.2f\n", total_wt / n)
return 0;
}
```

Result:

```
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017> cd "c:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017\" ; if ($?)
 { gcc multilevel.c -o multilevel } ; if ($?) { .\multilevel }
Enter number of processes: 4

P1
Arrival Time: 0
Burst Time: 4
Queue (1-System, 2-User): 1

P2
Arrival Time: 0
Burst Time: 3
Queue (1-System, 2-User): 1

P3
Arrival Time: 0
Burst Time: 8
Queue (1-System, 2-User): 2

P4
Arrival Time: 10
Burst Time: 5
Queue (1-System, 2-User): 1

Gantt Chart:
P1 P2 P1 P2 P3 P3 P3 P4 P4 P4 P3 P3 P3 P3 P3

PID     AT      BT      CT      TAT     WT
P1      0       4       6       6       2
P2      0       3       7       7       4
P3      0       8       20      20      12
P4      10      5       15      5       0

Average TAT = 9.50
Average WT = 4.50
```

Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:
c) Rate- Monotonic
d) Earliest-deadline First
e) Proportional scheduling

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
struct Process {
    int pid;
    int burst;
    int period;
    int deadline;
    int remaining;
    int current_deadline;
    int tickets;
};
int gcd(int a, int b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
int lcm(int a, int b) {
    if (a == 0 || b == 0)
        return 0;
    return (a * b) / gcd(a, b);
}
void runRMS(struct Process p[], int n, int hyperPeriod) {
    printf("\n--- Running Rate Monotonic Scheduling ---\n");
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].period > p[j + 1].period) {
```

```
                struct Process temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
    for (int t = 0; t < hyperPeriod; t++) {
        int chosen = -1;
        for (int i = 0; i < n; i++) {
            if (t % p[i].period == 0)
                p[i].remaining = p[i].burst;
        }
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0) {
                chosen = i;
                break;
            }
        }
        if (chosen != -1) {
            printf("Time %d: P%d\n", t, p[chosen].pid);
            p[chosen].remaining--;
        } else
            printf("Time %d: Idle\n", t);
    }
}
void runEDF(struct Process p[], int n, int hyperPeriod) {
    printf("\n--- Running Earliest Deadline First (Custom Deadlines) ---\n");
    for (int i = 0; i < n; i++) {
        p[i].remaining = p[i].burst;
        p[i].current_deadline = p[i].deadline;
    }
```

```
    for (int t = 0; t < hyperPeriod; t++) {
        for (int i = 0; i < n; i++) {
            if (t > 0 && t % p[i].period == 0) {
                if (p[i].remaining > 0) {
                    printf("[!] Deadline Miss: P%d at time %d\n", p[i].pid, t);
                }
                p[i].remaining = p[i].burst;
                p[i].current_deadline = t + p[i].deadline;
            }
        }
        int chosen = -1;
        int minDeadline = 1e9;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0) {
                if (p[i].current_deadline < minDeadline) {
                    minDeadline = p[i].current_deadline;
                    chosen = i;
                }
            }
        }
        if (chosen != -1) {
            printf("Time %dms: P%d (Abs Deadline: %d)\n", t, p[chosen].pid,
                   p[chosen].current_deadline);
            p[chosen].remaining--;
        } else {
            printf("Time %dms: Idle\n", t);
        }
    }
}
void runProportionalShare(struct Process p[], int n, int totalTime) {
    printf("\n--- Running Proportional Share (Lottery) ---\n");
    int totalTickets = 0;
```

```
    for (int i = 0; i < n; i++) {
        printf("Enter tickets for P%d: ", p[i].pid);
        scanf("%d", &p[i].tickets);
        totalTickets += p[i].tickets;
    }
    for (int t = 0; t < totalTime; t++) {
        int lottery = rand() % totalTickets;
        int sum = 0, chosen = -1;
        for (int i = 0; i < n; i++) {
            sum += p[i].tickets;
            if (lottery < sum) {
                chosen = i;
                break;
            }
        }
        printf("Time %d: P%d wins lottery\n", t, p[chosen].pid);
    }
}
int main() {
    int n, choice;
    printf("Enter the number of processes: ");
    if (scanf("%d", &n) != 1)
        return 1;
    struct Process p[n], temp_p[n];
    int hyperPeriod = 1;
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nProcess %d:\n", p[i].pid);
        printf("  Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("  Period: ");
        scanf("%d", &p[i].period);
```

```
        printf("  Relative Deadline: ");
        scanf("%d", &p[i].deadline);
        hyperPeriod = lcm(hyperPeriod, p[i].period);
    }
    while (1) {
        printf("\n--- Real-Time Scheduling Simulator ---\n");
        printf("1. Rate Monotonic (RMS)\n");
        printf("2. Earliest Deadline First (EDF)\n");
        printf("3. Proportional Share (Lottery)\n");
        printf("4. Exit\n");
        printf("Select an option: ");
        scanf("%d", &choice);
        if (choice == 4)
            break;
        for (int i = 0; i < n; i++)
            temp_p[i] = p[i];
        switch (choice) {
        case 1:
            runRMS(temp_p, n, hyperPeriod);
            break;
        case 2:
            runEDF(temp_p, n, hyperPeriod);
            break;
        case 3:
            runProportionalShare(temp_p, n, hyperPeriod);
            break;
        default:
            printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

Result:

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Enter the number of processes: 2

Process 1:
  Burst Time: 20
  Period: 50
  Relative Deadline: 0

Process 2:
  Burst Time: 35
  Period: 100
  Relative Deadline: 0

--- Real-Time Scheduling Simulator ---
1. Rate Monotonic (RMS)
2. Earliest Deadline First (EDF)
3. Proportional Share (Lottery)
4. Exit
Select an option: 1

--- Running Rate Monotonic Scheduling ---
Time 0: P1
Time 1: P1
Time 2: P1
Time 3: P1
Time 4: P1
Time 5: P1
Time 6: P1
Time 7: P1
Time 8: P1
Time 9: P1
Time 10: P1
```

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Enter the number of processes: 3

Process 1:
  Burst Time: 3
  Period: 20
  Relative Deadline: 7

Process 2:
  Burst Time: 2
  Period: 5
  Relative Deadline: 4

Process 3:
  Burst Time: 2
  Period: 10
  Relative Deadline: 8

--- Real-Time Scheduling Simulator ---
1. Rate Monotonic (RMS)
2. Earliest Deadline First (EDF)
3. Proportional Share (Lottery)
4. Exit
Select an option: 2

--- Running Earliest Deadline First (Custom Deadlines) ---
Time 0ms: P2 (Abs Deadline: 4)
Time 1ms: P2 (Abs Deadline: 4)
Time 2ms: P1 (Abs Deadline: 7)
Time 3ms: P1 (Abs Deadline: 7)
Time 4ms: P1 (Abs Deadline: 7)
Time 5ms: P3 (Abs Deadline: 8)
Time 6ms: P3 (Abs Deadline: 8)
Time 7ms: P2 (Abs Deadline: 9)
Time 8ms: P2 (Abs Deadline: 9)
Time 9ms: Idle
```

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Enter the number of Processes: 5

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

Process 4:
Tickets: 40

Process 5:
Tickets: 50

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 10
Tickets picked: 62, Winner: P4
Tickets picked: 96, Winner: P4
Tickets picked: 106, Winner: P5
Tickets picked: 41, Winner: P3
Tickets picked: 44, Winner: P3
Tickets picked: 28, Winner: P2
Tickets picked: 115, Winner: P5
Tickets picked: 34, Winner: P3
Tickets picked: 133, Winner: P5
Tickets picked: 115, Winner: P5

The Process: P1 gets 6.67% of Processor Time.

The Process: P2 gets 13.33% of Processor Time.

The Process: P3 gets 20.00% of Processor Time.

The Process: P4 gets 26.67% of Processor Time.

The Process: P5 gets 33.33% of Processor Time.
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017>
```

Program 4a:

Write a C program to simulate producer-consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int empty = BUFFER_SIZE;
int full = 0;
void produce() {
    if (empty == 0) {
        printf("\nBuffer is full! Cannot produce.\n");
        return;
    }
    int item;
    printf("Enter item to produce: ");
    scanf("%d", &item);
    buffer[in] = item;
    printf("Produced: %d\n", item, in);
    in = (in + 1) % BUFFER_SIZE;
    empty--;
    full++;
}
void consume() {
    if (full == 0) {
        printf("Buffer is empty! Cannot consume.\n");
        return;
    }
    int item = buffer[out];
    printf("Consumed: %d\n", item, out);
```

```
    out = (out + 1) % BUFFER_SIZE;
    full--;
    empty++;
}
int main() {
    int choice;
    printf("Buffer Size: %d\n", BUFFER_SIZE);
    while (1) {
        printf("\n1. Produce\n2. Consume\n3. Exit\n");
        printf("Current State: [Full slots: %d | Empty slots: %d]\n", full,
               empty);
        printf("Enter choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice) {
        case 1:
            produce();
            break;
        case 2:
            consume();
            break;
        case 3:
            printf("Exiting...\n");
            exit(0);
        default:
            printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

Result:

```
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017> cd "c:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017\"
Buffer Size: 5

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 0 | Empty slots: 5]
Enter choice: 1
Enter item to produce: 10
Produced: 10

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 1
Enter item to produce: 20
Produced: 20

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 2 | Empty slots: 3]
Enter choice: 1
Enter item to produce: 30
Produced: 30

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 3 | Empty slots: 2]
Enter choice: 2
Consumed: 10

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 2 | Empty slots: 3]
Enter choice: 2
Consumed: 20

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 3
Exiting...
PS C:\Users\BMSCECSE-L4\Documents\Aditya_A_1BM24CS017>
```

Program 4b:

Write a C program to simulate the concept of Dining Philosophers problem.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#define N 5              // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];
void *philosopher(void *num) {
   int id = *(int *)num;
   int left = id;           // left fork index
   int right = (id + 1) % N; // right fork index
   while (1) {
      printf("Philosopher %d is thinking.\n", id);
      sleep(1); // Thinking
      // To avoid deadlock, philosophers with even id pick left then right,
      // odd id pick right then left.
      if (id % 2 == 0) {
         // Pick up left fork
         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d.\n", id, left);
         // Pick up right fork
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);
      } else { // Pick up right fork
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id,
               right); // Pick up left
         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d.\n", id, left);
      }
```

```
        // Eating
        printf("Philosopher %d is eating.\n", id);
        sleep(2); // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
    return NULL;
}
int main() {
    int i;
    int ids[N];
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}
```

Result:

```
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017> cd "c:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017\"
_p } ; if ($?) { .\dining_p }
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 0 picked up left fork 0.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
```

Program 5a:

Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
#include <stdbool.h>
#define MAX_PROCESSES 10
#define MAX_RESOURCES 10
int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[MAX_PROCESSES][MAX_RESOURCES];
    int max[MAX_PROCESSES][MAX_RESOURCES];
    int need[MAX_PROCESSES][MAX_RESOURCES];
    int available[MAX_RESOURCES];
    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }
    printf("\nEnter Max Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
    printf("\nEnter Available Resources:\n");
    for (int j = 0; j < m; j++) {
        scanf("%d", &available[j]);
```

```
}
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}
bool finish[MAX_PROCESSES] = {false};
int safeSequence[MAX_PROCESSES];
int work[MAX_RESOURCES];
// Work = Available
for (int j = 0; j < m; j++) {
    work[j] = available[j];
}
int count = 0;
while (count < n) {
    bool found = false;
    for (int i = 0; i < n; i++) {
        if (finish[i] == false) {
            bool canExecute = true;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    canExecute = false;
                    break;
                }
            }
            if (canExecute) {
                for (int j = 0; j < m; j++) {
                    work[j] += allocation[i][j];
                }
                safeSequence[count] = i;
                count++;
```

```
                finish[i] = true;
                found = true;
            }
        }
    }
    if (!found) {
        break;
    }
}
if (count == n) {
    printf("\nSystem is in SAFE state.\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < n; i++) {
        printf("P%d", safeSequence[i]);
        if (i != n - 1) {
            printf(" -> ");
        }
    }
    printf("\n");
}
else {
    printf("\nSystem is NOT in safe state.\n");
}
return 0;
}
```

Result:

```
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017> cd "c:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017\"
 .\bankers }
 Enter number of processes: 5
 Enter number of resource types: 3

 Enter Allocation Matrix:
 0 1 0
 2 0 0
 3 0 2
 2 1 1
 0 0 2

 Enter Max Matrix:
 7 5 3
 3 2 2
 9 0 2
 2 2 2
 4 3 3

 Enter Available Resources:
 3 3 2

 System is in SAFE state.
 Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017>
```

Program 5b:

Deadlock Detection

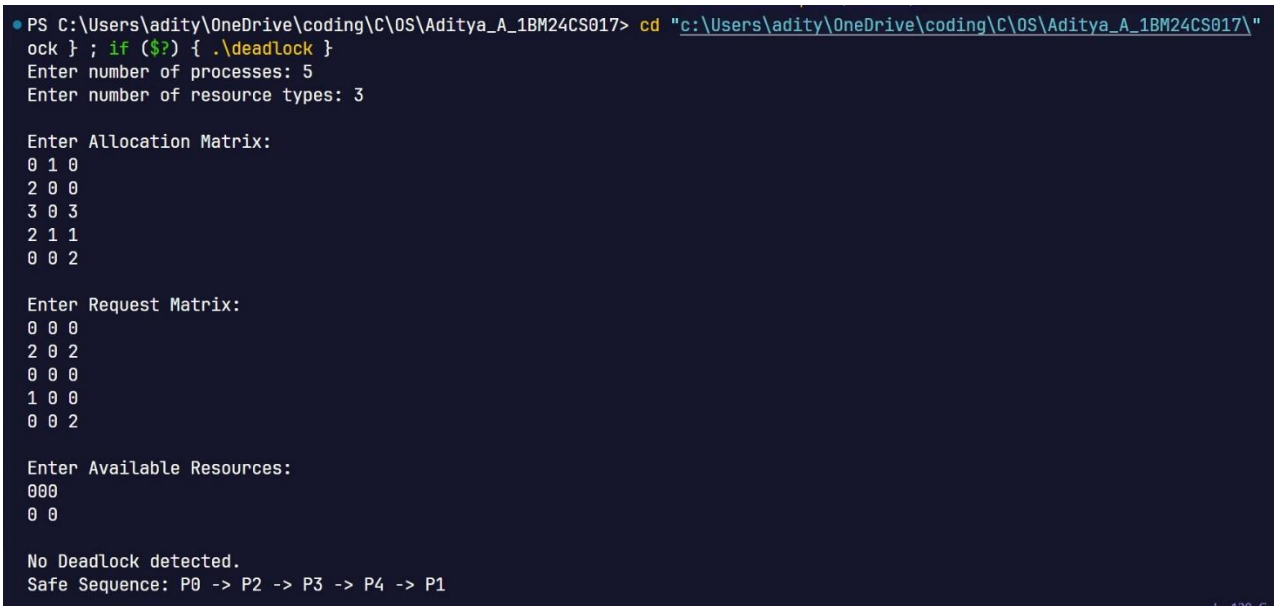
```
#include <stdio.h>
#include <stdbool.h>
#define MAX_PROCESSES 10
#define MAX_RESOURCES 10
int main() {
   int n, m;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resource types: ");
   scanf("%d", &m);
   int allocation[MAX_PROCESSES][MAX_RESOURCES];
   int request[MAX_PROCESSES][MAX_RESOURCES];
   int available[MAX_RESOURCES]
   printf("\nEnter Allocation Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         scanf("%d", &allocation[i][j]);
      }
   }
   printf("\nEnter Request Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         scanf("%d", &request[i][j]);
      }
   }
   printf("\nEnter Available Resources:\n");
   for (int j = 0; j < m; j++) {
      scanf("%d", &available[j]);
   }
```

```
bool finish[MAX_PROCESSES];
int work[MAX_RESOURCES];
int safeSequence[MAX_PROCESSES];
for (int j = 0; j < m; j++) {
   work[j] = available[j];
}
for (int i = 0; i < n; i++) {
   bool zeroAllocation = true;
   for (int j = 0; j < m; j++) {
      if (allocation[i][j] != 0) {
         zeroAllocation = false;
         break;
      }
   }
   finish[i] = zeroAllocation;
}
int count = 0;
while (1) {
   bool found = false;
   for (int i = 0; i < n; i++) {
      if (finish[i] == false) {
         bool canExecute = true;
         for (int j = 0; j < m; j++) {
            if (request[i][j] > work[j]) {
               canExecute = false;
               break;
            }
         }
         if (canExecute) {
            for (int j = 0; j < m; j++) {
               work[j] += allocation[i][j];
```

```
                finish[i] = true;
                safeSequence[count] = i;
                count++;
                found = true;
            }
        }
    }
    if (!found) {
        break;
    }
}
bool deadlock = false;
for (int i = 0; i < n; i++) {
    if (finish[i] == false) {
        deadlock = true;
        break;
    }
}
if (!deadlock) {
    printf("\nNo Deadlock detected.\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < count; i++) {
        printf("P%d", safeSequence[i]);
        if (i != count - 1) {
            printf(" -> ");
        }
    }
    printf("\n");
}
else {
    printf("\nDeadlock detected.\n");
    printf("Processes involved in deadlock: ");
```

```
        for (int i = 0; i < n; i++) {
            if (finish[i] == false) {
                printf("P%d ", i);
            }
        printf("\n");
    }
    return 0;
}
```

Result:



Program 6:

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit

b) Best-fit

c) First-fit

```
#include <stdio.h>
#define MAX 100
void printTable(char name[], int allocation[], int processes[], int n) {
    printf("\n===== %s =====\n", name);
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
void firstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    int tempBlocks[MAX];
    for (int i = 0; i < m; i++)
        tempBlocks[i] = blocks[i];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processes[i]) {
                allocation[i] = j;
                tempBlocks[j] -= processes[i];
```

```
            break;
        }
    }
}
printTable("FIRST FIT", allocation, processes, n);
}
void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    int tempBlocks[MAX];
    for (int i = 0; i < m; i++)
        tempBlocks[i] = blocks[i];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int best = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processes[i]) {
                if (best == -1 || tempBlocks[j] < tempBlocks[best]) {
                    best = j;
                }
            }
        }
        if (best != -1) {
            allocation[i] = best;
            tempBlocks[best] -= processes[i];
        }
    }
    printTable("BEST FIT", allocation, processes, n);
}
void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    int tempBlocks[MAX];
```

```
    for (int i = 0; i < m; i++)
        tempBlocks[i] = blocks[i];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processes[i]) {
                if (worst == -1 || tempBlocks[j] > tempBlocks[worst]) {
                    worst = j;
                }
            }
        }
        if (worst != -1) {
            allocation[i] = worst;
            tempBlocks[worst] -= processes[i];
        }
    }
    printTable("WORST FIT", allocation, processes, n);
}
int main() {
    int m, n;
    int blocks[MAX], processes[MAX];
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter sizes of processes:\n");
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &processes[i]);
    firstFit(blocks, m, processes, n);
    bestFit(blocks, m, processes, n);
    worstFit(blocks, m, processes, n);
    return 0;
}
```

Result:

```
PS C:\Users\adity\OneDrive\coding\C\OS> cd "c:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017\"
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

===== FIRST FIT =====
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             2
4               426             Not Allocated

===== BEST FIT =====
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

===== WORST FIT =====
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             5
4               426             Not Allocated
PS C:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017>
```

Program 7:

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>
int main() {
    int pages[20], frames[10];
    int n, f, i, j, k, faults;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    // ---------------- FIFO ----------------
    printf("\nFIFO Page Replacement\n");
    for (i = 0; i < f; i++)
        frames[i] = -1;
    faults = 0;
    int index = 0;
    for (i = 0; i < n; i++)
        int found = 0;
        for (j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
```

```
            frames[index] = pages[i];
            index = (index + 1) % f;
            faults++;
        }
    }
    printf("Page Faults = %d\n", faults);
    // ---------- LRU ----------
    printf("\nLRU Page Replacement\n");
    int time[10], count = 0;
    for (i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;
    }
    faults = 0;
    for (i = 0; i < n; i++) {
        int found = 0;
        for (j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                count++;
                time[j] = count;
                found = 1;
                break;
            }
        }
        if (!found) {
            int min = 0;
            for (j = 1; j < f; j++) {
                if (time[j] < time[min])
                    min = j;
            }
            frames[min] = pages[i];
            count++;
```

```
            time[min] = count;
            faults++;
        }
    }
    printf("Page Faults = %d\n", faults);
    // ---------- Optimal ----------
    printf("\nOptimal Page Replacement\n");
    for (i = 0; i < f; i++)
        frames[i] = -1;
    faults = 0;
    for (i = 0; i < n; i++) {
        int found = 0;
        for (j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            int pos = -1;
            // Empty frame
            for (j = 0; j < f; j++) {
                if (frames[j] == -1) {
                    pos = j;
                    break;
                }
            }
            // Find farthest page
            if (pos == -1) {
                int farthest = -1;
                for (j = 0; j < f; j++) {
                    int nextUse = 999;
```

```
            for (k = i + 1; k < n; k++) {
                if (frames[j] == pages[k]) {
                    nextUse = k;
                    break;
                }
            }
            if (nextUse > farthest) {
                farthest = nextUse;
                pos = j;
            }
        }
    }
    frames[pos] = pages[i];
    faults++;
  }
  printf("Page Faults = %d\n", faults);
  return 0;
}
```

Result:

```
PS C:\Users\adity\OneDrive\coding\C\OS\Operating-Systems\program7> cd "c:\Users\adity\OneDrive\coding\C\OS\Aditya_A_1BM24CS017\"
ge } ; if ($?) { .\page }
Enter number of pages: 21
Enter page reference string: 5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0
Enter number of frames: 3

FIFO Page Replacement
Page Faults = 11

LRU Page Replacement
Page Faults = 12

Optimal Page Replacement
Page Faults = 9
```